Quality is never an accident; it is always the result of high intention, sincere effort, intelligent direction and skillful execution; it represents the wise choice of many alternatives.

—William A. Foster

It would appear that we have reached the limits of what it is possible to achieve with computer technology, although one should be careful with such statements, as they tend to sound pretty silly in 5 years.

—John von Neumann, 1949

CHAPTER 9 Alternative Architectures

9.1 INTRODUCTION

Our previous chapters have featured excursions into the background of computing technology. The presentations have been distinctly focused on uniprocessor systems from a computer science practitioner's point of view. We hope that you have gained an understanding of the functions of various hardware components and can see how each contributes to overall system performance. This understanding is vital not only to hardware design, but also to efficient algorithm implementation. Most people gain familiarity with computer hardware through their experiences with personal computers and workstations. This leaves one significant area of computer architecture untouched: that of alternative architectures. Therefore, the focus of this chapter is to introduce you to a few of the architectures that transcend the classical von Neumann approach.

This chapter discusses RISC machines, architectures that exploit instruction-level parallelism, and multiprocessing architectures (with a brief introduction to parallel processing). We begin with the notorious RISC versus CISC debate to give you an idea of the differences between these two ISAs and their relative advantages and disadvantages. We then provide a taxonomy by which the various architectures may be classified, with a view as to how the parallel architectures fit into the classification. Next, we consider topics relevant to instruction-level parallel architectures, emphasizing superscalar architectures and reintroducing EPIC (explicitly parallel instruction computers) and VLIW (very long instruction word) designs. Finally, we provide a brief introduction to multiprocessor systems and some alternative approaches to parallelism.

Computer hardware designers began to reevaluate various architectural principles in the early 1980s. The first target of this reevaluation was the instruction set architec-

ture. The designers wondered why a machine needed an extensive set of complex instructions when only about 20% of the instructions were used most of the time. This question led to the development of RISC machines, which we first introduced in Chapters 4 and 5, and to which we now devote an entire section of this chapter. The pervasiveness of RISC designs has led to a unique marriage of CISC with RISC. Many architectures now employ RISC cores to implement CISC architectures.

Chapters 4 and 5 described how new architectures, such as VLIW, EPIC, and multiprocessors, are taking over a large percentage of the hardware market. The invention of architectures exploiting instruction-level parallelism has led to techniques that accurately predict the outcome of branches in program code before the code is executed. Prefetching instructions based on these predictions has greatly increased computer performance. In addition to predicting the next instruction to fetch, high degrees of instruction-level parallelism have given rise to ideas such as speculative execution, where the processor guesses the value of a result before it has actually been calculated.

The subject of alternative architectures also includes multiprocessor systems. For these architectures, we return to the lesson we learned from our ancestors and the friendly ox. If we are using an ox to pull out a tree, and the tree is too large, we don't try to grow a bigger ox. Instead, we use two oxen. Multiprocessing architectures are analogous to the oxen. We need them if we are to budge the stumps of intractable problems. However, multiprocessor systems present us with unique challenges, particularly with respect to cache coherence and memory consistency.

We note that although some of these alternative architectures are taking hold, their real advancement is dependent upon their incremental cost. Currently, the relationship between the performance delivered by advanced systems and their cost is nonlinear, with the cost far exceeding performance gains in most situations. This makes it cost prohibitive to integrate these architectures into mainstream applications. Alternative architectures do have their place in the market, however. Highly numerical science and engineering applications demand machines that outperform standard uniprocessor systems. For computers in this league, cost is usually not an issue.

As you read this chapter, keep in mind the previous computer generations introduced in Chapter 1. Many people believe that we have entered a new generation based on these alternative architectures, particularly in the area of parallel processing.

9.2 RISC MACHINES

We introduced RISC architectures in the context of instruction set design in Chapters 4 and 5. Recall that RISC machines are so named because they originally offered a smaller instruction set as compared to CISC machines. As RISC machines were being developed, the term "reduced" became somewhat of a misnomer, and is even more so now. The original idea was to provide a set of minimal instructions that could carry out all essential operations: data movement, ALU operations, and branching. Only explicit `load` and `store` instructions were permitted access to memory.

Complex instruction set designs were motivated by the high cost of memory. Having more complexity packed into each instruction meant that programs could be smaller, thus occupying less storage. CISC ISAs employ variable-length instructions, which keeps the simple instructions short, while also allowing for longer, more complicated instructions. Additionally, CISC architectures include a large number of instructions that directly access memory. So what we have at this point is a dense, powerful, variable-length set of instructions, which results in a varying number of clock cycles per instruction. Some complex instructions, particularly those instructions that access memory, require hundreds of cycles. In certain circumstances, computer designers have found it necessary to slow down the system clock (making the interval between clock ticks larger) to allow sufficient time for instructions to complete. This all adds up to longer execution time.

Human languages exhibit some of the qualities of RISC and CISC and serve as a good analogy to understand the differences between the two. Suppose you have a Chinese pen pal. Let's assume each of you speak and write fluently in both English and Chinese. You both wish to keep the cost of your correspondence at a minimum, although you both enjoy sharing long letters. You have a choice between using expensive airmail paper, which will save considerable postage, or using plain paper and paying extra for stamps. A third alternative is to pack more information onto each written page.

As compared to the Chinese language, English is simple but verbose. Chinese characters are more complex than English words, and what might require 200 English letters might require only 20 Chinese characters. Corresponding in Chinese requires fewer symbols, saving on both paper and postage. However, reading and writing Chinese requires more effort, because each symbol contains more information. The English words are analogous to RISC instructions, whereas the Chinese symbols are analogous to CISC instructions. For most English-speaking people, "processing" the letter in English would take less time, but would also require more physical resources.

Although many sources tout RISC as a new and revolutionary design, its seeds were sown in the mid-1970s through the work of IBM's John Cocke. Cocke started building his experimental Model 801 mainframe in 1975. This system initially received little attention, its details being disclosed only many years later. In the interim, David Patterson and David Ditzel published their widely acclaimed "Case for a Reduced Instruction Set Computer" in 1980. This paper gave birth to a radically new way of thinking about computer architecture, and brought the acronyms *CISC* and *RISC* into the lexicon of computer science. The new architecture proposed by Patterson and Ditzel advocated simple instructions, all of the same length. Each instruction would perform less work, but the time required for instruction execution would be constant and predictable.

Support for RISC machines came by way of programming observations on CISC machines. These studies revealed that data movement instructions accounted for approximately 45% of all instructions, ALU operations (including arithmetic, comparison, and logical) accounted for 25%, and branching (or flow control) amounted to 30%. Although many complex instructions existed, few were used. This finding, coupled with the advent of cheaper and more plentiful

memory, and the development of VLSI technology, led to a different type of architecture. Cheaper memory meant that programs could use more storage. Larger programs consisting of simple, predictable instructions could replace shorter programs consisting of complicated and variable length instructions. Simple instructions would allow the use of shorter clock cycles. In addition, having fewer instructions would mean fewer transistors were needed on the chip. Fewer transistors translated to cheaper manufacturing costs and more chip real estate available for other uses. Instruction predictability coupled with VLSI advances would allow various performance-enhancing tricks, such as pipelining, to be implemented in hardware. CISC does not provide this diverse array of performance-enhancement opportunities.

We can quantify the differences between RISC and CISC using the basic computer performance equation as follows:

$$\frac{\text{time}}{\text{program}} = \frac{\text{time}}{\text{cycle}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{instructions}}{\text{program}}$$

Computer performance, as measured by program execution time, is directly proportional to clock cycle time, the number of clock cycles per instruction, and the number of instructions in the program. Shortening the clock cycle, when possible, results in improved performance for RISC as well as CISC. Otherwise, CISC machines increase performance by reducing the number of instructions per program. RISC computers minimize the number of cycles per instruction. Yet both architectures can produce identical results in approximately the same amount of time. At the gate level, both systems perform an equivalent quantity of work. So what's going on between the program level and the gate level?

CISC machines rely on microcode to tackle instruction complexity. Microcode tells the processor how to execute each instruction. For performance reasons, microcode is compact, efficient, and it certainly must be correct. Microcode efficiency, however, is limited by variable length instructions, which slow the decoding process, and a varying number of clock cycles per instruction, which makes it difficult to implement instruction pipelines. Moreover, microcode interprets each instruction as it is fetched from memory. This additional translation process takes time. The more complex the instruction set, the more time it takes to look up the instruction and engage the hardware suitable for its execution.

RISC architectures take a different approach. Most RISC instructions execute in one clock cycle. To accomplish this speedup, microprogrammed control is replaced by hardwired control, which is faster at executing instructions. This makes it easier to do instruction pipelining, but more difficult to deal with complexity at the hardware level. In RISC systems, the complexity removed from the instruction set is pushed up a level into the domain of the compiler.

To illustrate, let's look at an instruction. Suppose we want to compute the product, 5×10. The code on a CISC machine might look like this:

```
mov ax, 10
mov bx, 5
mul bx, ax
```

A minimalistic RISC ISA has no multiplication instructions. Thus, on a RISC system our multiplication problem would look like this:

```
        mov ax, 0
        mov bx, 10
        mov cx, 5
Begin:  add ax, bx
        loop Begin   ;causes a loop cx times
```

The CISC code, although shorter, requires more clock cycles to execute. Suppose that on each architecture, register-to-register moves, addition, and loop operations each consume one clock cycle. Suppose also that a multiplication operation requires 30 clock cycles.[1] Comparing the two code fragments we have:

CISC instructions:

$$\begin{aligned}\text{Total clock cycles} &= (2\ \texttt{movs} \times 1 \text{ clock cycle}) + (1\ \texttt{mul} \times 30 \text{ clock cycles}) \\ &= 32 \text{ clock cycles}\end{aligned}$$

RISC instructions:

$$\begin{aligned}\text{Total clock cycles} &= (3\ \texttt{movs} \times 1 \text{ clock cycle}) + (5\ \texttt{adds} \times 1 \text{ clock cycle}) \\ &\quad + (5\ \texttt{loops} \times 1 \text{ clock cycle}) \\ &= 13 \text{ clock cycles}\end{aligned}$$

Add to this the fact that RISC clock cycles are often shorter than CISC clock cycles, and it should be clear that even though there are more instructions, the actual execution time is less for RISC than for CISC. This is the main inspiration behind the RISC design.

We have mentioned that reducing instruction complexity results in simpler chips. Transistors formerly employed in the execution of CISC instructions are used for pipelines, cache, and registers. Of these three, registers offer the greatest potential for improved performance, so it makes sense to increase the number of registers and to use them in innovative ways. One such innovation is the use of *register window sets*. Although not as widely accepted as other innovations associated with RISC architectures, register windowing is nevertheless an interesting idea and is briefly introduced here.

High-level languages depend on modularization for efficiency. Procedure calls and parameter passing are natural side effects of using these modules. Calling a procedure is not a trivial task. It involves saving a return address, preserving register values, passing parameters (either by pushing them on a stack or using registers), branching to the subroutine, and executing the subroutine. Upon subroutine completion, parameter value modifications must be saved, and previous register values must be restored before returning execution to the calling program. Saving registers, passing parameters, and restoring registers involves considerable effort and resources. With RISC chips having the capacity for hundreds

[1]This is not an unrealistic number—a multiplication on an Intel 8088 requires 133 clock cycles for two 16-bit numbers.

RISC	CISC
Multiple register sets, often consisting of more than 256 registers	Single register set, typically 6 to 16 registers total
Three register operands allowed per instruction (e.g., `add R1, R2, R3`)	One or two register operands allowed per instruction (e.g., `add R1, R2`)
Parameter passing through efficient on-chip register windows	Parameter passing through inefficient off-chip memory
Single-cycle instructions (except for `load` and `store`)	Multiple-cycle instructions
Hardwired control	Microprogrammed control
Highly pipelined	Less pipelined
Simple instructions that are few in number	Many complex instructions
Fixed length instructions	Variable length instructions
Complexity in compiler	Complexity in microcode
Only load and store instructions can access memory	Many instructions can access memory
Few addressing modes	Many addressing modes

TABLE 9.1 The Characteristics of RISC Machines versus CISC Machines

data streams that flow into the processor. A machine can have either one or multiple streams of data, and can have either one or multiple processors working on this data. This gives us four possible combinations: *SISD* (*single instruction stream, single data stream*), *SIMD* (*single instruction stream, multiple data streams*), *MISD* (*multiple instruction streams, single data stream*), and *MIMD* (*multiple instruction streams, multiple data streams*).

Uniprocessors are SISD machines. SIMD machines, which have a single point of control, execute the same instruction simultaneously on multiple data values. The SIMD category includes array processors, vector processors, and systolic arrays. MISD machines have multiple instruction streams operating on the same data stream. MIMD machines, which employ multiple control points, have independent instruction and data streams. Multiprocessors and most current parallel systems are MIMD machines. SIMD computers are simpler to design than MIMD machines, but they are also considerably less flexible. All of the SIMD multiprocessors must execute the *same* instruction simultaneously. If you think about this, executing something as simple as conditional branching could quickly become very expensive.